

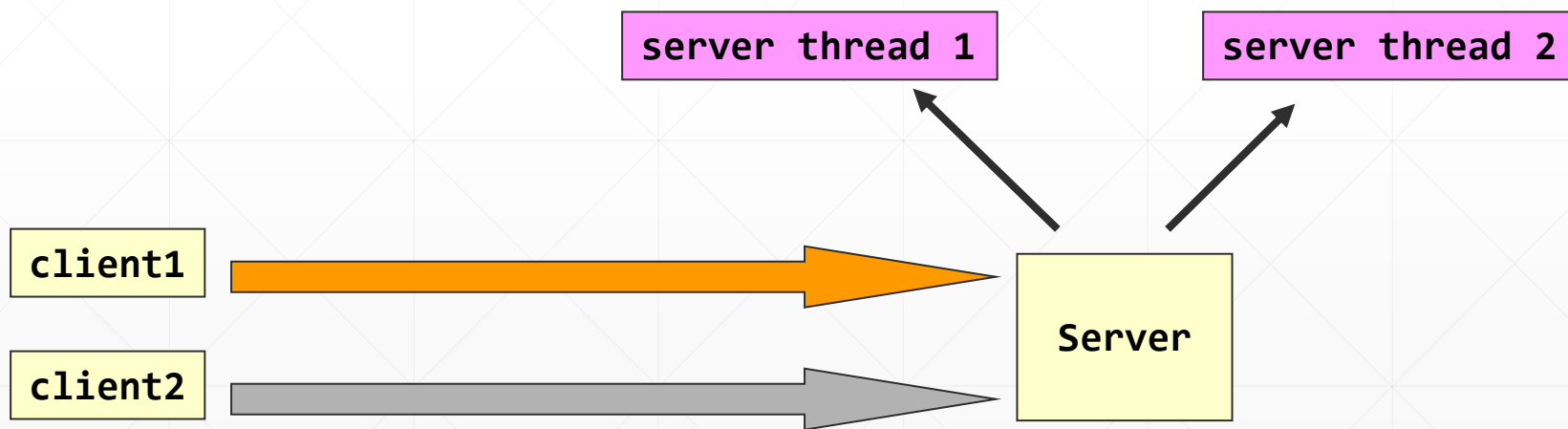


开发多客户端Socket网络应用

北京理工大学计算机学院
金旭亮

服务端如何处理多个连接请求？

处理方法：将服务器写成多线程的，不同的处理线程为不同的客户端服务。主线程只负责循环等待，当有一个客户端连接请求时，新建一个线程，由此线程负责接收客户输入的信息。



使用多线程处理客户连接-1

可以将响应单个客户端连接要完成的工作移到一个独立的线程中，如右图所示：

```
//在一个独立的线程中向客户端发送数据
1 usage
private static class DateTimeThread extends Thread {
    4 usages
    private final Socket clientConnection;
    1 usage
    public DateTimeThread(Socket connection) { this.clientConnection = connection; }
    @Override
    public void run() {
        try {
            System.out.println("接收到客户端连接: " + clientConnection);
            System.out.println("负责处理的线程: " + Thread.currentThread());
            var out = new BufferedWriter(new OutputStreamWriter(
                clientConnection.getOutputStream(),
                StandardCharsets.UTF_8));
            Date now = new Date();
            String message=now.toString();
            out.write(message);
            out.newLine();
            out.flush();
            System.out.println("写入数据: "+message);

        } catch (Exception e) {
        } finally {
            try {
                clientConnection.close(); //数据发送完毕, 关闭连接
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

使用多线程处理客户连接-2

```
//向客户端返回Server端的时间,  
//使用多线程处理客户连接  
public class DayTimeServer2 {  
    2 usages  
    public final static int PORT = 8899;  
    public static void main(String[] args) {  
        try (ServerSocket server = new ServerSocket(PORT)) {  
            System.out.println("正在监听端口" + PORT + ",等待客户端连接.....");  
            while (true) {  
                Socket clientConnection = server.accept();  
                //启用新线程响应客户端的连接请求  
                new DateTimeThread(clientConnection).start();  
                //线程启动之后, 监听线程继续调用accept()方法等待新的连接请求  
            }  
        } catch (IOException e) {  
            System.out.println(e);  
        }  
    }  
}
```

使用这种编程模式，Server 为每个客户端连接都创建一个新的线程响应，这样就能保证Server能及时地响应新的连接。

```
DayTimeServer2 x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" --module-path  
正在监听端口8899,等待客户端连接.....  
接收到客户端连接: Socket[addr=/127.0.0.1,port=9450,localport=8899]  
负责处理的线程: Thread[Thread-0,5,main]  
写入数据: Tue Jun 07 18:35:08 CST 2022
```

存在的问题



由于操作系统创建线程是有开销的，比如要为每个线程提供线程堆栈，如果在某个时间段时并发连接太多导致大量的线程被创建，很有可能耗尽服务器的可用内存，从而让服务器停止服务。

不管客户端有没有数据传来，只要客户端没有断开连接，线程就必须阻塞等待，由于线程不能及时地关闭，会影响到服务器的运行。

上述第一个问题，可以通过引入一个线程池来解决。
第二个问题，可以使用NIO来解决。

Java提供的线程池



使用线程池处理客户连接

```
public class DayTimeServer3 {  
    2 usages  
    public final static int PORT = 8899;  
    public static void main(String[] args) {  
        //创建一个有50个线程对象的线程池  
        ExecutorService pool = Executors.newFixedThreadPool( nThreads: 50);  
        try (ServerSocket server = new ServerSocket(PORT)) {  
            System.out.println("正在监听端口" + PORT + ",等待客户端连接.....");  
            while (true) {  
                Socket clientConnection = server.accept();  
                //将完成数据传送工作的任务对象传给线程池去执行  
                Callable<Void> task = new DateTimeTask(clientConnection);  
                pool.submit(task);  
            }  
        } catch (IOException e) {  
            System.out.println(e);  
        }  
    }  
}  
  
//此类负责向客户端发送特定的数据  
1 usage  
private static class DateTimeTask implements Callable<Void> {  
    4 usages  
    private Socket clientConnection;  
    1 usage  
    public DateTimeTask(Socket connection) { this.clientConnection = connection; }  
    @Override  
    public Void call() {...}  
}
```

通过使用线程池，能够有效地避免操作系统一次创建太多的线程，从而提升了服务器响应并发请求的能力。

在实际开发中，推荐使用线程池而不是直接new线程对象的方式来响应客户端请求。

```
Run: DayTimeServer3 x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" --module-path  
正在监听端口8899,等待客户端连接.....  
接收到客户端连接: Socket[addr=/127.0.0.1,port=3819,localport=8899]  
负责处理的线程: Thread[pool-1-thread-1,5,main]  
写入数据: Tue Jun 07 18:39:40 CST 2022
```